

チーム 23 ハッカソン 1

基本設計・詳細設計

梅澤颯太（学籍番号：25G1021）

2026 年 4 月 29 日

目次

1	基本設計	2
1.1	117：音色合成 基本設計	2
2	詳細設計	3
2.1	124：Arduino からの NOTE 受信と発音の仕組み	3
2.2	125：4 パート発音管理	3
2.3	126：金管楽器風音色の具体実装	3
2.4	127：Python による FFT 解析	4
3	生成 AI の利用について	4

1 基本設計

1.1 117：音色合成 基本設計

1.1.1 目的

Processing を使って、金管楽器らしい音色を作る方針を決める。「倍音の重ね合わせ（加算合成）」と「振幅エンベロープ（音の大きさの時間変化）」を組み合わせる。

1.1.2 音色の仕組み

楽器音の音色は以下の 2 つで決まる。

- **倍音の含み方**：基音（出したい音の周波数）の整数倍の周波数成分をいくつ、どの強さで重ねるか、偶数倍音・奇数倍音の有無や強さが音色の違いに大きく影響する。
- **倍音の立ち上がり方（エンベロープ）**：音が鳴り始めてから消えるまでの音量の変化。楽器らしさはここで決まることが多い。

WavetableGenerator.gen10() を使って倍音の振幅を指定する。

1.1.3 倍音構成の方針

金管楽器の特徴は「倍音が多く、明るくて力強い音」である。具体的には、基音（第 1 倍音）から第 2・第 3 倍音あたりが特に強く、高次の倍音になるほど徐々に弱くなる。

表 1 金管楽器風の倍音構成（案）

倍音次数	振幅の大きさ（0.0～1.0）	根拠・メモ
第 1 倍音（基音）	1.00	一番大きい成分
第 2 倍音	0.70	金管では 2 倍音が強い
第 3 倍音	0.50	明るさを加える
第 4 倍音	0.30	厚みを加える
第 5 倍音	0.20	少しブライトにする
第 6 倍音	0.10	高次はだんだん弱く
第 7 倍音	0.05	ごくわずかに含む

1.1.4 エンベロープの方針

授業で使った HackInstrument クラスでは、Line を使って「音が鳴り始めから時間をかけて音量をゼロに下げていく（減衰）」という処理をしている。金管楽器らしくするためには、さらに「鳴り始め（立ち上がり）」も加えたい。

立ち上がり（Attack） 音が鳴り始めてから最大音量に達するまでの時間。約 20 ミリ秒（とても短い）。

減衰・持続（Decay / Sustain） 最大音量から少し落ち着いた音量で安定する。息を吹き続けている感じ。

余韻（Release） 息を止めて音が消えるまでの時間。約 60 ミリ秒。

ただし、Minim の Line で表現できるのは「1 本の直線的な変化」のみである。そのため音の出来次第で ADSR クラスを使う方針とする。

2 詳細設計

2.1 124：Arduino からの NOTE 受信と発音の仕組み

2.1.1 概要

Arduino から Serial 通信で届く音の指令を Processing が受け取り，対応する音を鳴らす．通信フォーマットの詳細は 115 と 130 が確定してから決める．ここでは大まかな流れのみ示す．

2.1.2 処理の流れ（案）

1. Arduino が Serial 通信で「どの音を・どの長さで・どの大きさに鳴らすか」という情報を Processing に送る．
2. Processing の `serialEvent()` 関数が自動で呼ばれ，データを受け取る．
3. 受け取ったデータから音名・長さ・振幅を取り出す．
4. 取り出した情報をもとに `HackInstrument` を生成し，`out.playNote()` で音を鳴らす．

2.2 125：4 パート発音管理

4 つの楽器パート（現段階では金管楽器）をそれぞれ独立して鳴らせるように管理する仕組みを作る．授業では，`melody`・`duration`・`startTime` のような配列を使って音の情報を管理する方法を学んだ．同じ発想で，パートごとの音色（Waveform）を配列で管理する．

2.3 126：金管楽器風音色の具体実装

2.3.1 概要

タスク 117 で決めた倍音構成とエンベロープを，Processing のコードに実際に落とし込む．

2.3.2 倍音テーブルの実装（案）

授業で使った `WavetableGenerator.gen10()` にそのまま値を入れる．

```
1 Waveform makeBrassWaveform() {
2   return WavetableGenerator.gen10(
3     4096, // サンプル数 (の累乗にすること) 2
4     new float[] {
5       1.00f, // 第倍音 (基音): 最大 1
6       0.70f, // 第倍音: 金管は倍音が強い 22
7       0.50f, // 第倍音 3
8       0.30f, // 第倍音 4
9       0.20f, // 第倍音 5
10      0.10f, // 第倍音 6
11      0.05f // 第倍音: ほんの少し 7
12    }
13  );
14 }
```

Listing 1 金管楽器風の波形生成（案）

2.3.3 エンベロープの実装 (案)

既存の HackInstrument クラスの noteOn() 内で ampEnv.activate(duration, maxAmp, 0) を呼ぶ。これにより、音符の長さ全体をかけて最大音量からゼロへ直線的に減衰する。鳴り始めが最大音量で、そこから徐々に消えていく形になる。Line は一度に一本の直線変化しか持てないため、立ち上がり (Attack) の表現は行わない。(音の出来次第で ADSR クラスを使う)

表 2 ampEnv.activate() に渡す値

時間	開始音量	終了音量
duration (音符の長さ全体)	maxAmp	0.0

2.3.4 パラメータ調整の方針

表 1・表 2 の値はいずれも仮置きである。実際に音を鳴らしながら耳で確認し、金管楽器らしく聞こえるよう調整する。127 (FFT 解析) の結果が得られた場合はその値に更新する。

2.4 127: Python による FFT 解析

2.4.1 概要

実際の金管楽器の音声ファイルを Python で周波数分析 (FFT) し、倍音の強さとエンベロープの時定数を数値として取り出す。取り出した値を表 1 および attackTime に反映する。

2.4.2 手順 (案)

- 1. フリー素材から金管楽器の単音サンプルを入手する。
- 2. Python の scipy.fft または librosa で FFT を実施し、各倍音成分のパワーを求める。
- 3. 基音のパワーを 1.0 として正規化し、第 2〜第 7 倍音の比率を計算する。
- 4. 音量の時間変化 (RMS カーブ) から立ち上がり時間を推定する。
- 5. 得られた値を makeBrassWaveform() の配列と attackTime に反映する。

3 生成 AI の利用について

本レポートを作成するにあたり、ClaudeAI を利用しました。生成された内容を吟味しつつ、修正・加筆して完成させました。